

Heart Disease Risk Predictor

An End-to-End MLOps Pipeline

Assignment 01

Production-Grade ML Lifecycle

AIMLCZG523 — MLOps
BITS Pilani (WILP)

Author: Mohanraj G

BIT ID: 2024AC05048

Date: 10 July 2026

Repository: https://github.com/gmohanraj73/gmr_sem3_aimlczg523

Contents

1. Project Overview	4
High-Level Architecture	5
2. Setup & Installation	6
Prerequisites	6
Option A — pip + venv	6
Option B — conda	6
Reproducing the model (one command chain)	6
Repository Structure	8
Technology Stack	9
3. Exploratory Data Analysis	10
Data Quality & Missing Values	10
Class Balance	11
Feature Distributions & Correlation	11
4. Model Development	14
Feature Engineering & Preprocessing	14
Training Protocol	14
Candidate Models & Hyperparameter Search	14
ROC Curves & Confusion Matrices	16
Winner: Logistic Regression	17
Feature Importance	18
6. Experiment Tracking with MLflow	19
7. CI/CD Pipeline	22
8. Containerisation & API Design	23
FastAPI Service	23
Dockerfile Highlights	23
9. Kubernetes Deployment	25
Deployment	25
Service	25
10. Monitoring & Conclusion	27
Observability Stack	27
Lessons Learned	29

Future Work 29

Conclusion 30

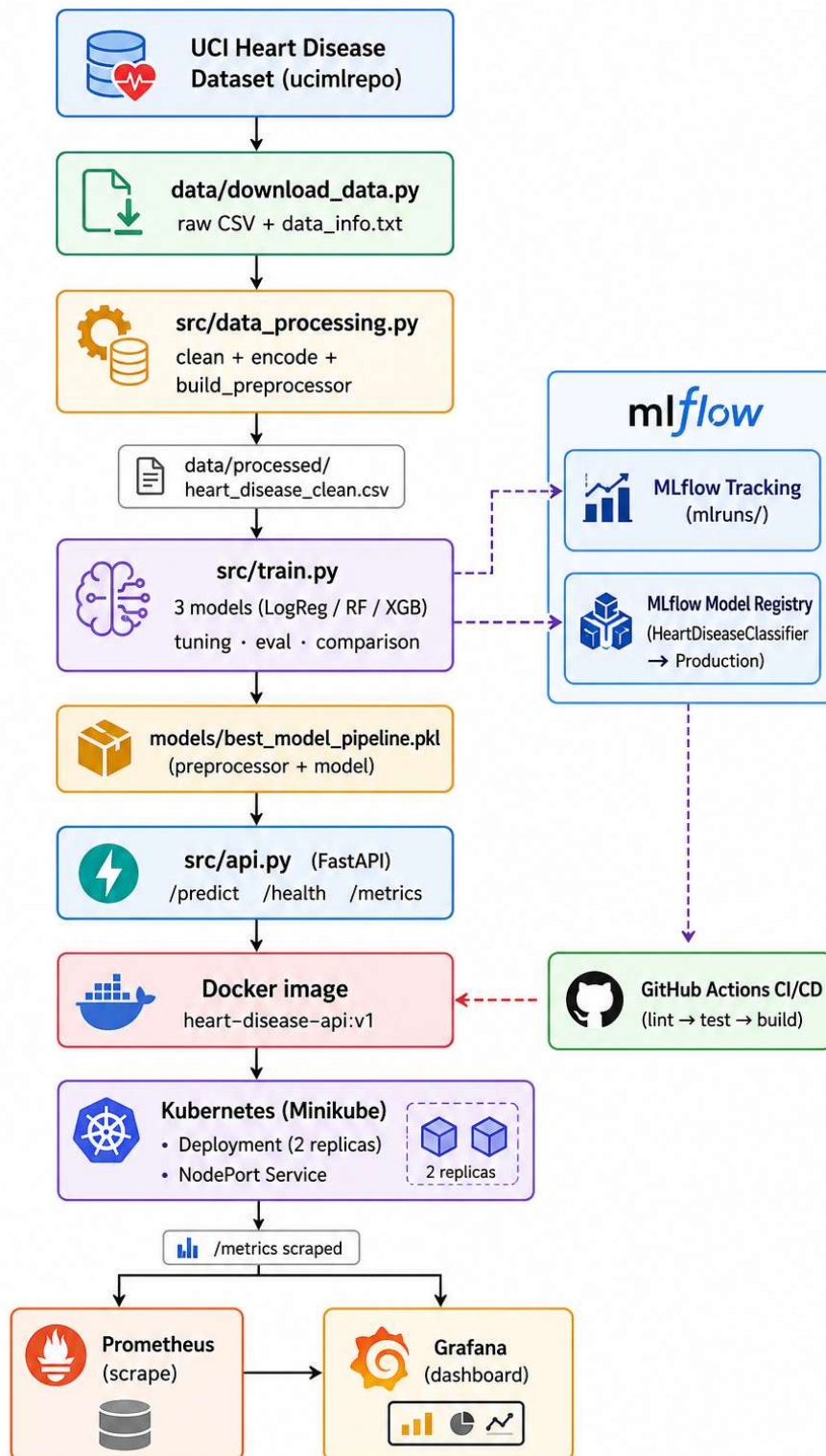
1. Project Overview

Cardiovascular disease is the leading cause of death worldwide, and early risk stratification materially improves clinical outcomes. This project builds a supervised machine-learning classifier that predicts whether a patient is likely to have heart disease from 13 routinely-collected clinical measurements (age, resting blood pressure, cholesterol, chest-pain type, maximum heart rate, and others), and — critically — wraps that model in a complete, reproducible MLOps lifecycle rather than leaving it as a notebook.

The problem is framed as binary classification: the original UCI severity grade (0–4) is collapsed to 0 = no disease and 1 = disease present. The deliverable is not merely an accurate model but a cloud-ready, monitored, continuously-tested prediction service. The pipeline spans data acquisition and cleaning, exploratory analysis, model development with hyperparameter tuning, experiment tracking and a model registry (MLflow), automated testing and CI/CD (GitHub Actions), containerisation (Docker), orchestration (Kubernetes / Minikube), and observability (Prometheus + Grafana).

A central design decision is that the winning model is persisted as a single self-contained scikit-learn Pipeline that embeds the fitted preprocessor together with the estimator. Inference therefore requires no external feature-engineering code, eliminating training/serving skew — the most common source of silent production failures in ML systems.

High-Level Architecture



2. Setup & Installation

The project runs from a clean environment using pinned dependencies. Two install paths are provided so the pipeline is reproducible on any machine.

Prerequisites

- **Python 3.10 and Git** — required for the core pipeline.
- **Docker Desktop** — required for containerisation and the monitoring stack.
- **Minikube + kubectl** — required for the Kubernetes deployment phase.

Option A — pip + venv

```
python -m venv venv  
venv\Scripts\activate      # Windows  
pip install -r requirements.txt -r requirements-dev.txt
```

Option B — conda

```
conda env create -f environment.yml  
conda activate mlops-heart-disease
```

Reproducing the model (one command chain)

```
python data/download_data.py # ->  
data/raw/heart_disease_raw.csv  
python src/data_processing.py # ->  
data/processed/heart_disease_clean.csv  
python -m src.train          # -> models/best_model_pipeline.pkl +  
mlruns/
```

All randomness is seeded (`random_state = 42`) across the train/test split, cross-validation folds, and every estimator, so results are deterministic and reproducible run-to-run.

Repository Structure

heart-disease-mlops/

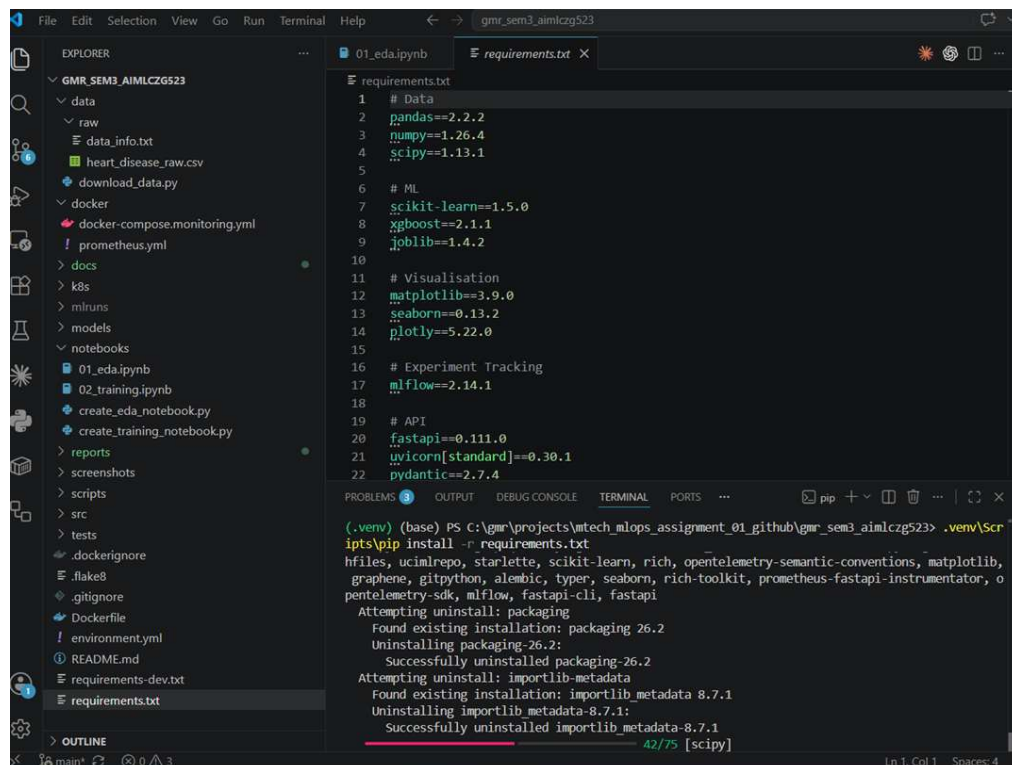
- └─ data/ raw + processed datasets, download_data.py
- └─ notebooks/ 01_eda.ipynb, 02_training.ipynb
- └─ src/ data_processing.py, train.py, predict.py, api.py
- └─ tests/ test_data.py, test_model.py, test_api.py
- └─ models/ best_model_pipeline.pkl, model_info.json
- └─ mlruns/ MLflow tracking store + registry
- └─ scripts/ test_inference.py, load_test.py
- └─ docker/ docker-compose.monitoring.yml,

prometheus.yml

- └─ k8s/ deployment.yaml, service.yaml
- └─ screenshots/ eda / mlflow / cicd / docker / k8s /

monitoring

- └─ .github/workflows/ ci.yml
- └─ Dockerfile
- └─ requirements.txt / requirements-dev.txt / environment.yml



The screenshot shows a VS Code editor with the Explorer view on the left displaying the repository structure. The main editor area shows the `requirements.txt` file with the following content:

```
1 # Data
2 pandas==2.2.2
3 numpy==1.26.4
4 scipy==1.13.1
5
6 # ML
7 scikit-learn==1.5.0
8 xgboost==2.1.1
9 joblib==1.4.2
10
11 # Visualisation
12 matplotlib==3.9.0
13 seaborn==0.13.2
14 plotly==5.22.0
15
16 # Experiment Tracking
17 mlflow==2.14.1
18
19 # API
20 fastapi==0.111.0
21 uvicorn[standard]==0.30.1
22 pydantic==2.7.4
```

The terminal window at the bottom shows the output of the command `pip install -r requirements.txt` in a virtual environment. The output indicates that several packages are being uninstalled and replaced with newer versions:

```
(.venv) (base) PS C:\gmr\projects\mtech_mlops_assignment_01_github\gmr_sem3_aimlczg523> .venv\Scripts\pip install -r requirements.txt
Found existing installation: packaging 26.2
Uninstalling packaging-26.2:
  Successfully uninstalled packaging-26.2
Attempting uninstall: importlib-metadata
Found existing installation: importlib-metadata 8.7.1
Uninstalling importlib-metadata-8.7.1:
  Successfully uninstalled importlib-metadata-8.7.1
```


Technology Stack

Layer	Tools
Language	Python 3.10
Data / ML	pandas, numpy, scikit-learn, XGBoost
Experiment tracking	MLflow (tracking + model registry)
API	FastAPI, Uvicorn, Pydantic
Testing / lint	pytest, pytest-cov, flake8, ruff, black
CI/CD	GitHub Actions
Container / Orchestration	Docker, Kubernetes (Minikube)
Monitoring	Prometheus, Grafana, prometheus-fastapi-instrumentator

3. Exploratory Data Analysis

The dataset is the UCI Heart Disease (Cleveland) collection, fetched programmatically via the `ucimlrepo` package (`id = 45`). It contains 303 patient records with 13 clinical features and one target column (`num`).

Property	Value
Rows × columns	303 × 14 (13 features + target)
Numeric features	age, trestbps, chol, thalach, oldpeak
Categorical features	sex, cp, fbs, restecg, exang, slope, ca, thal
Missing values	ca: 4, thal: 2 (imputed)
Target (binarised)	164 no-disease / 139 disease

Data Quality & Missing Values

The raw feed is clean of the '?' sentinel that plagues some Cleveland variants; only two columns carry genuine NaNs — `ca` (4 missing) and `thal` (2 missing). These are imputed with the column median (numeric) or mode (categorical) inside `DataProcessor.handle_missing`, so no rows are discarded and the full 303 records are retained for modelling.

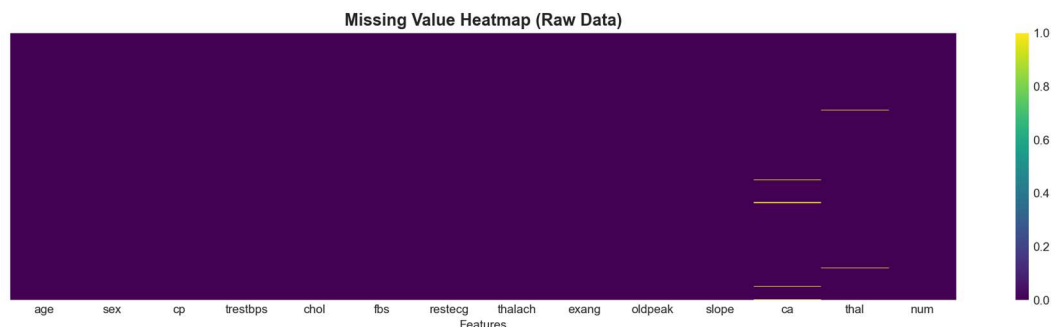


Figure 2. Missing-value heatmap — sparse missingness confined to `ca` and `thal`.

Class Balance

After binarising the severity grade, the classes are 164 no-disease (54%) versus 139 disease (46%). This is a mild imbalance well within the tolerance of the chosen algorithms, so no resampling (SMOTE / class weighting) was applied; a stratified split preserves the ratio in both train and test folds.

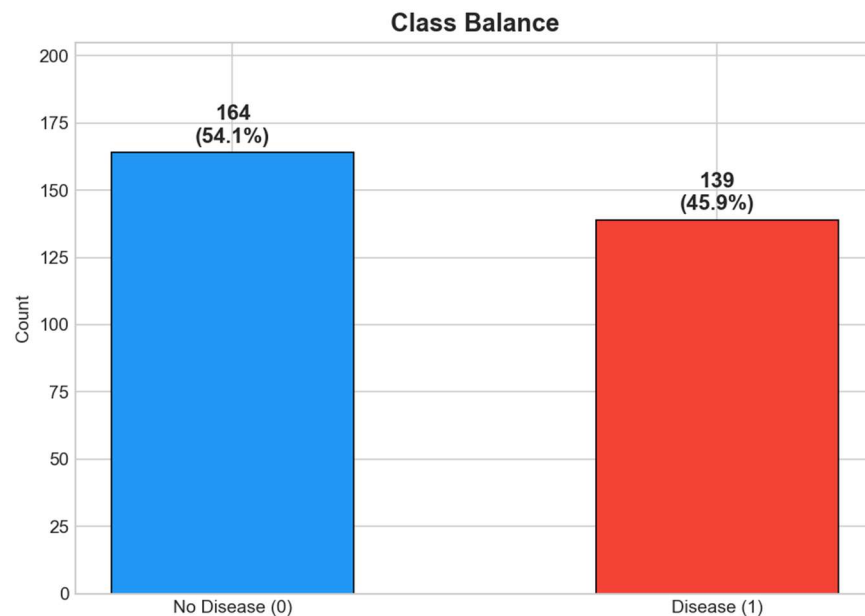


Figure 3. Target class distribution (0 = no disease, 1 = disease).

Feature Distributions & Correlation

Distributions of the five numeric features, split by target class, show clear separation for thalach (maximum heart rate — lower in diseased patients) and oldpeak (ST depression — higher in diseased patients). The correlation analysis confirms this: the features most associated with the target are chest-pain type (cp), thalassemia (thal), number of major vessels (ca), exercise-induced angina (exang), and oldpeak.

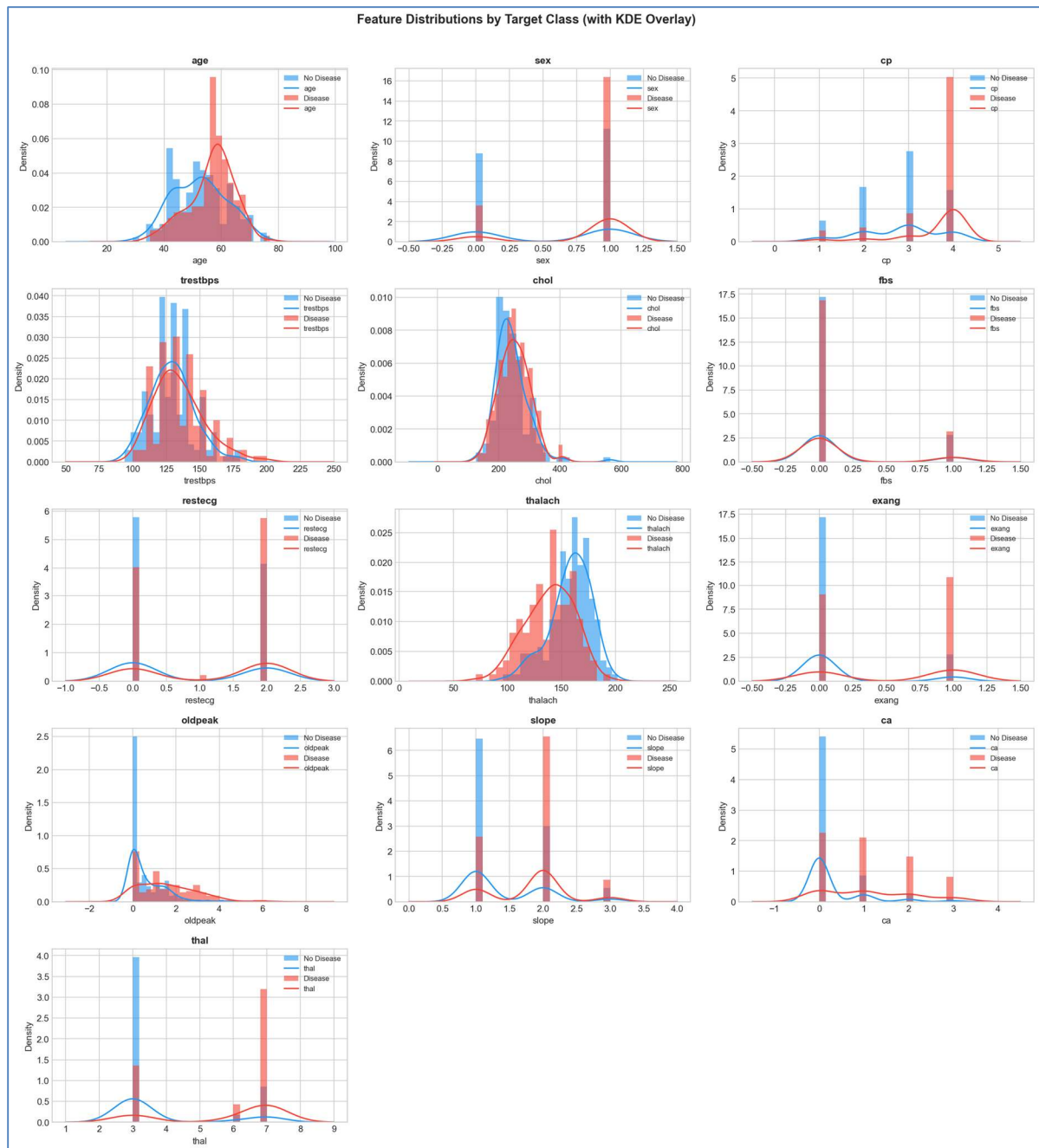


Figure 4. Numeric feature distributions coloured by target class.

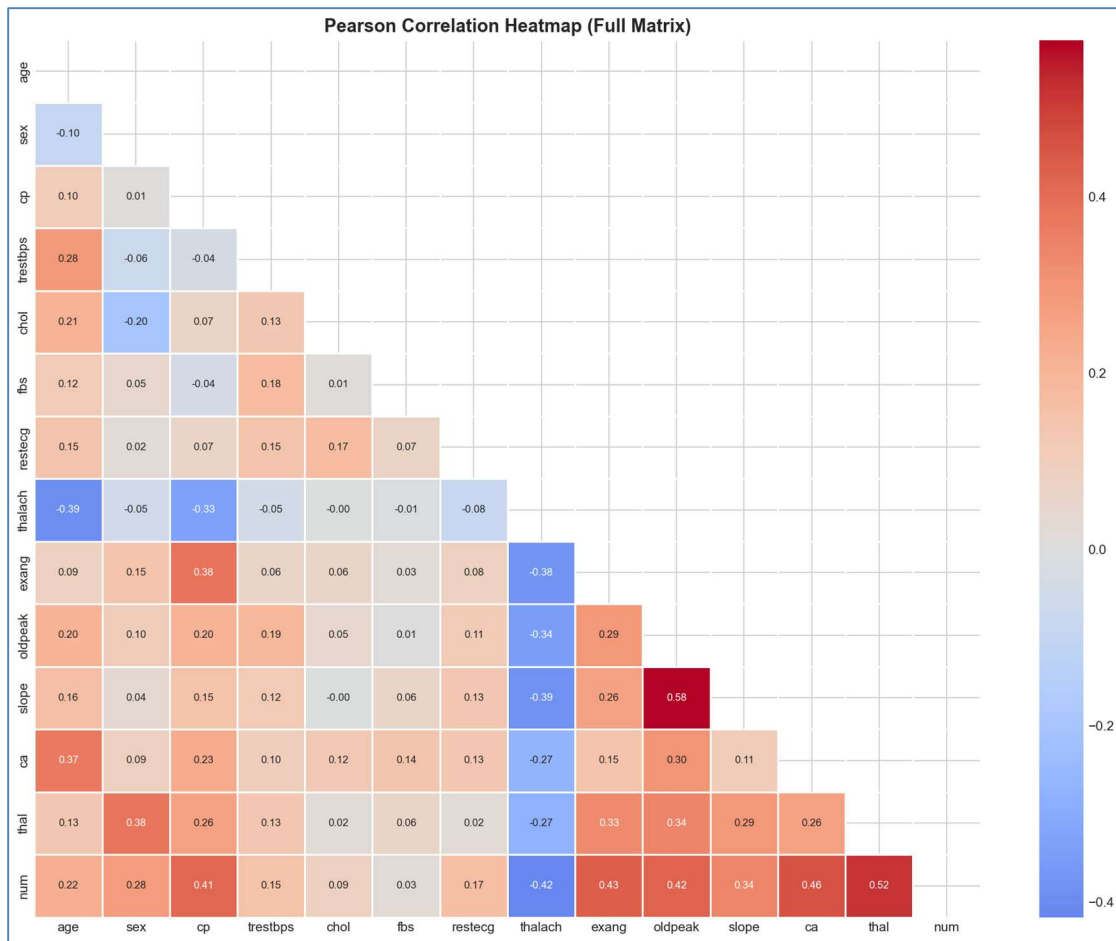


Figure 5. Pearson correlation heatmap across all features and the target.

4. Model Development

Feature Engineering & Preprocessing

Preprocessing is implemented as a scikit-learn ColumnTransformer so it is fitted only on training data and travels with the model:

- **Numeric columns** (age, trestbps, chol, thalach, oldpeak) are standardised with StandardScaler to zero mean and unit variance.
- **Categorical columns** (sex, cp, fbs, restecg, exang, slope, ca, thal) are one-hot encoded with handle_unknown='ignore', so unseen categories at inference degrade gracefully instead of crashing.

The transformer is composed with each estimator inside a single Pipeline object: Pipeline([('preprocessor', ...), ('model', ...)]). This is the crux of the packaging strategy — the persisted artifact contains both the fitted scaler/encoder and the trained estimator, so raw patient dictionaries can be scored directly with no separate preprocessing step.

Training Protocol

- **Split** — stratified 80/20 train/test (242 train, 61 test rows), random_state=42.
- **Cross-validation** — 5-fold StratifiedKFold, scored on ROC-AUC.
- **Selection metric** — ROC-AUC on the held-out test set, chosen because it is threshold-independent and robust to the mild class imbalance.

Candidate Models & Hyperparameter Search

Three complementary algorithms were tuned — a linear baseline, a bagging ensemble, and a boosting ensemble:

Model	Tuner	Search space
Logistic Regression	GridSearchCV (exhaustive)	$C \in \{0.01, 0.1, 1, 10\}$; penalty $\in \{l1, l2\}$;

		solver $\in$ {liblinear, lbfgs}
Random Forest	RandomizedSearchCV (15 iters)	n_estimators $\in$ {100...500}; max_depth $\in$ {None,4,6,8,12}; min_samples_split $\in$ {2,5,10}
XGBoost	RandomizedSearchCV (15 iters)	n_estimators $\in$ {100...500}; max_depth $\in$ {3,4,6,8}; learning_rate $\in$ {0.01...0.2}; subsample $\in$ {0.6,0.8,1.0}

Every search operates over the full Pipeline (hyperparameters prefixed with model__), guaranteeing that preprocessing is re-fitted inside each CV fold and no information leaks from validation folds into training.

5. Model Comparison & Winner Selection

All three tuned pipelines were evaluated on the identical 61-row test set. Results below are from the reproducible training run (seed 42):

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV Mean
Logistic Regression ★	0.8689	0.8333	0.8929	0.8621	0.9589	0.9070
Random Forest	0.8689	0.8333	0.8929	0.8621	0.9481	0.9017
XGBoost	0.8852	0.8621	0.8929	0.8772	0.9437	0.8846

ROC Curves & Confusion Matrices

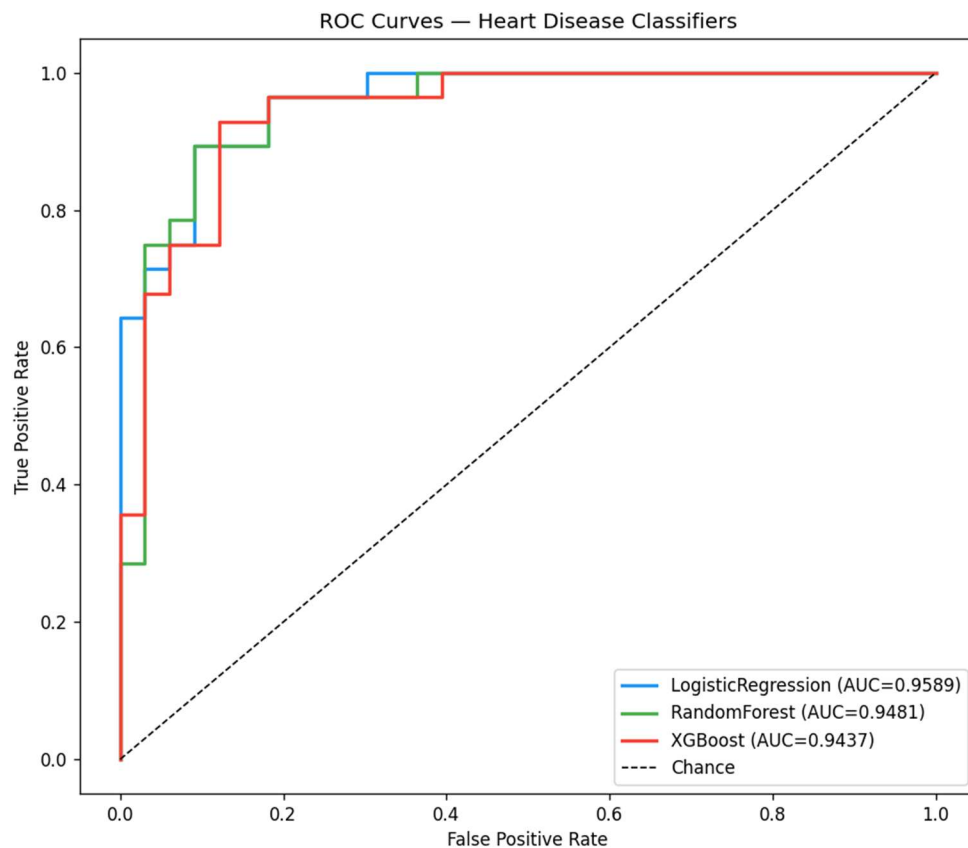


Figure 6. ROC curves — Logistic Regression achieves the highest AUC (0.9589).

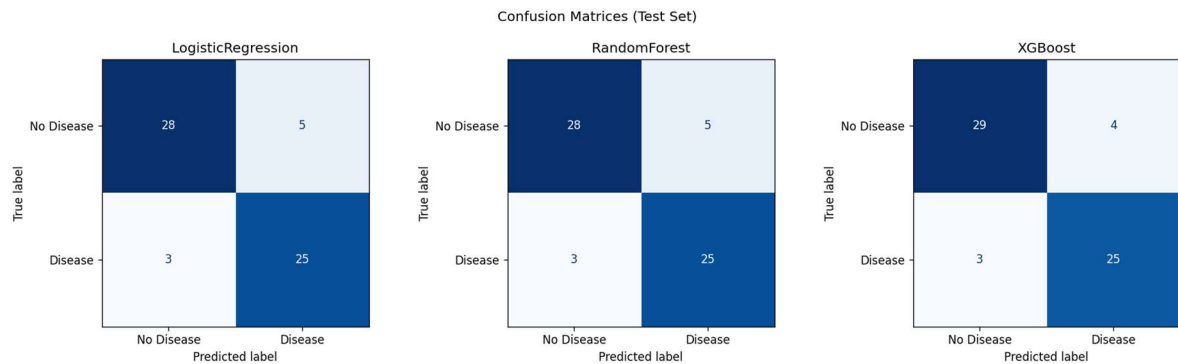


Figure 7. Confusion matrices on the test set for all three candidates.

Winner: Logistic Regression

Logistic Regression ($C=1.0$, $\text{penalty}=l1$, $\text{solver}=liblinear$) is selected as the production model. Although XGBoost edges ahead on raw accuracy (0.8852), Logistic Regression wins on the primary selection metric, ROC-AUC (0.9589 vs 0.9437), and also posts the best cross-validation mean (0.9070), indicating the strongest and most stable ranking ability across thresholds.

Beyond the numbers, the choice is well-justified for a clinical decision-support setting: the dataset is small (303 rows), where a regularised linear model generalises better and is far less prone to overfitting than deep tree ensembles; the L1 penalty adds implicit feature selection; and coefficient-based models are interpretable, which matters for medical trust and auditability. It is also the lightest artifact (≈ 4.5 KB) and lowest-latency option to serve.

Feature Importance

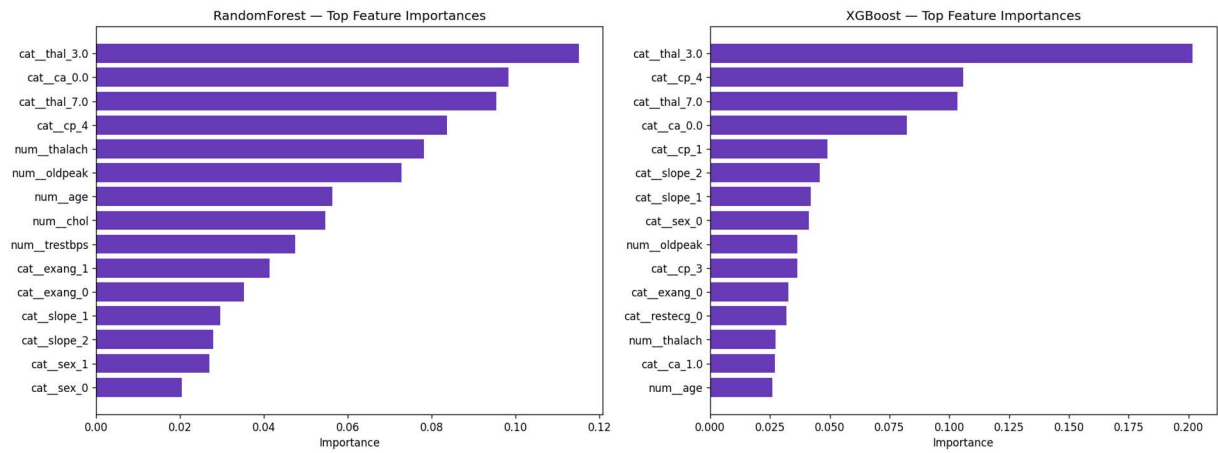


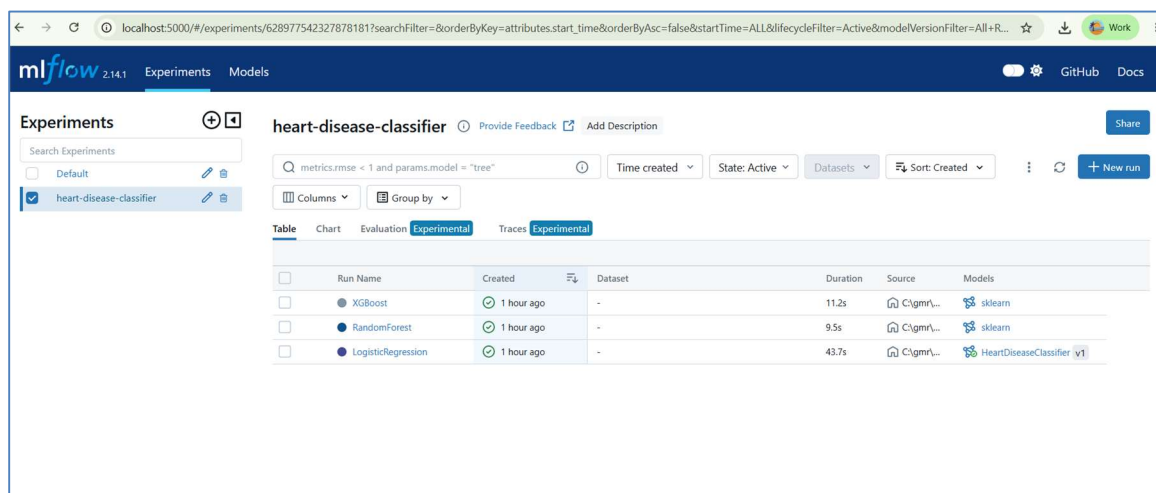
Figure 8. Top feature importances from the tree ensembles, corroborating the EDA: cp, thal, ca, oldpeak, and thalach dominate.

6. Experiment Tracking with MLflow

Every training run is tracked with MLflow using a local file store (file:./mlruns) under a single experiment, heart-disease-classifier. Each of the three models is logged as its own run capturing:

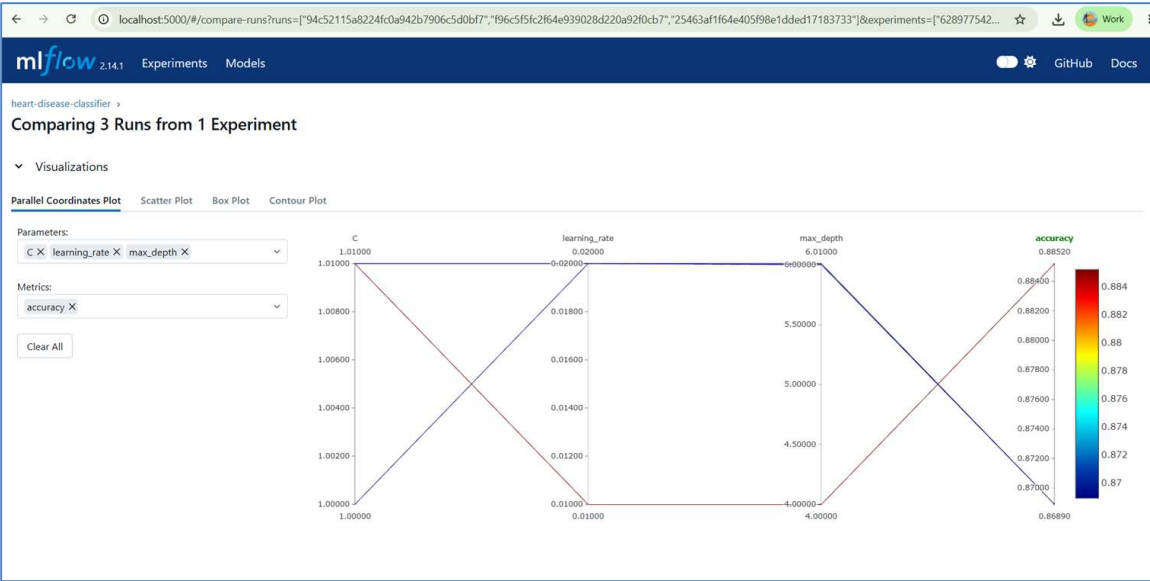
- **Parameters** — the full set of tuned hyperparameters for the winning configuration.
- **Metrics** — accuracy, precision, recall, F1, ROC-AUC, and CV mean/std.
- **Artifacts** — the ROC-curve, confusion-matrix, and feature-importance plots.
- **Model** — the complete fitted Pipeline via `mlflow.sklearn.log_model`.

The best run is additionally tagged (status = production-candidate, dataset = heart-disease-uci, author = Mohanraj G). The winning model is then registered in the MLflow Model Registry as HeartDiseaseClassifier and promoted to the Production stage, giving a clean separation between experimentation and the blessed serving artifact.



Run Name	Created	Dataset	Duration	Source	Models
XGBoost	1 hour ago	-	11.2s	C:\gmr\...	sklearn
RandomForest	1 hour ago	-	9.5s	C:\gmr\...	sklearn
LogisticRegression	1 hour ago	-	43.7s	C:\gmr\...	HeartDiseaseClassifier v1

Experiments List



Run details

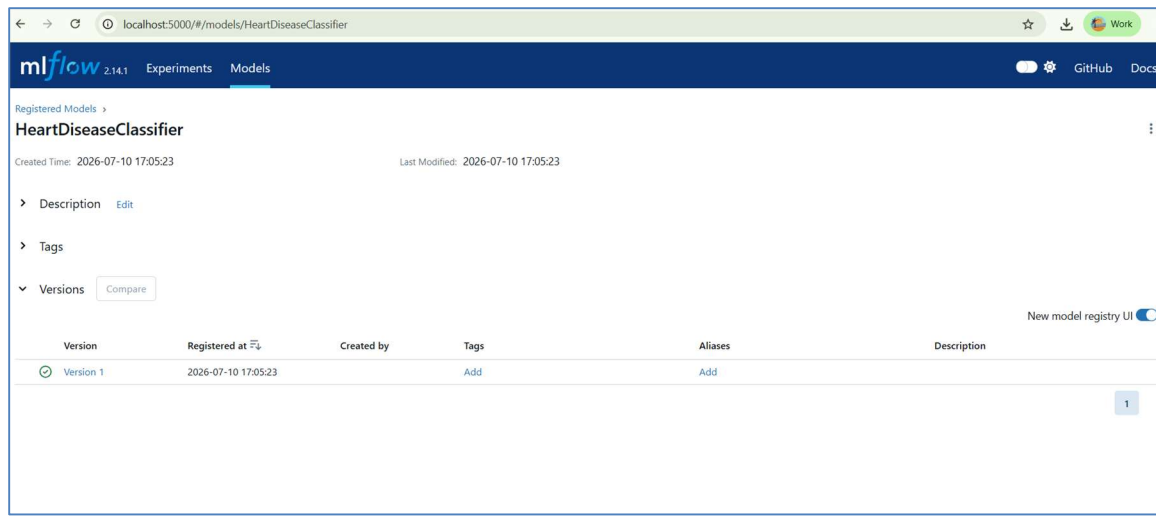
Run ID:	94c52115a8224fc0a942b7906c5d0bf7	f96c5f5fc2f64e939028d220a92f0cb7	25463af1f64e405f98e1dded17183733
Run Name:	XGBoost	RandomForest	LogisticRegression
Start Time:	2026-07-10 17:05:12	2026-07-10 17:05:02	2026-07-10 17:04:19
End Time:	2026-07-10 17:05:23	2026-07-10 17:05:12	2026-07-10 17:05:02
Duration:	11.2s	9.5s	43.7s

Parameters

Show diff only

C	1.0
learning_rate	0.01
max_depth	4
min_samples_split	5
n_estimators	200

Comparison



Model Registry

Launch the UI to reproduce these views:

```
mlflow ui --backend-store-uri ./mlruns --port 5000 #  
http://localhost:5000
```

7. CI/CD Pipeline

Continuous integration is implemented with GitHub Actions (`.github/workflows/ci.yml`) and triggers on every push and pull request to the main and dev branches. The workflow is composed of three sequential, gated jobs — each depends on the previous succeeding, so a failure short-circuits the pipeline and blocks the merge.

Job	What it does	Fails loudly when...
lint	Runs flake8 and ruff over src/ and tests/	any style or lint violation is found
test	Installs deps, downloads data, trains the model, runs pytest with coverage	any of the 21 unit tests fails
build-docker	Builds the Docker image and smoke-tests /health in a running container	the image fails to build or the container is unhealthy

This ordering enforces a fast-feedback contract: cheap static checks run first, then correctness tests, then the expensive container build — so obvious mistakes are caught in seconds rather than minutes. The 'fail loudly' requirement is satisfied because each tool returns a non-zero exit code on violation, which GitHub Actions surfaces as a red, merge-blocking check.

8. Containerisation & API Design

FastAPI Service

The model is served by a FastAPI application (src/api.py) exposing four endpoints:

Endpoint	Purpose
GET /	Service metadata and a pointer to the docs.
GET /health	Liveness/readiness probe: {status, model_loaded}.
POST /predict	Scores one patient; returns prediction, probability, risk_level.
GET /metrics	Prometheus-format metrics (auto-instrumented).

Requests are validated by a Pydantic model (PatientFeatures) whose field bounds follow the original UCI encoding the model was trained on (cp 1–4, restecg 0–2, slope 1–3, ca 0–3, thal ∈ {3,6,7}); malformed or out-of-range input is rejected automatically with HTTP 422. The predictor is loaded once as a process-wide singleton at startup, so the first real request is not penalised by model-load latency.

Example request and response:

POST /predict

```
{"age":67,"sex":1,"cp":4,"trestbps":120,"chol":229,"fbs":0,"restecg":2,
"thalach":129,"exang":1,"oldpeak":2.6,"slope":2,"ca":2,"thal":7}

→ {"prediction":1,"probability":0.9865,"risk_level":"High",
"model_version":"1.0.0","latency_ms":31.07}
```

Dockerfile Highlights

- **Slim base** — python:3.10-slim keeps the image small.

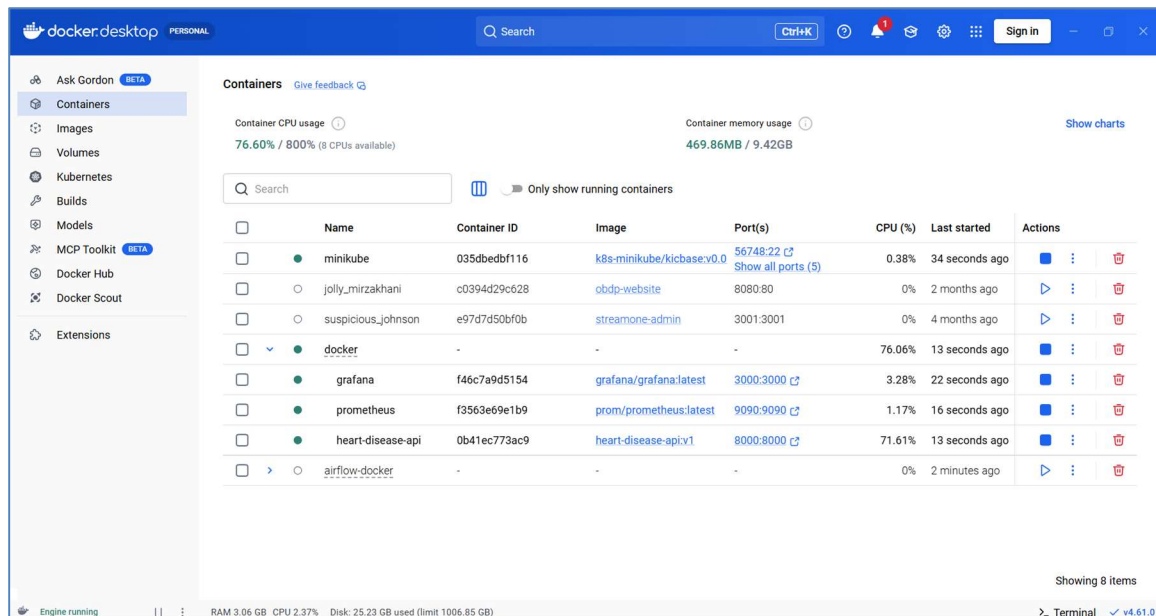
- **Layer caching** — requirements are copied and installed before application code, so dependency layers are re-used across code changes.
- **Security** — runs as a non-root appuser.
- **Self-healing** — a container HEALTHCHECK curls /health every 30s so the orchestrator can detect and restart unhealthy replicas.

```
(base) PS C:\Users\Mohanraj G> cd c:\gmr\projects\mtech_mlops_assignment_01_github\bits_sem3_mlops_001
(base) PS C:\gmr\projects\mtech_mlops_assignment_01_github\bits_sem3_mlops_001> docker build -t heart-disease-api:v1 .
[+] Building 3.8s (13/13) FINISHED
=> [internal] load build definition from Dockerfile                                0.1s
=> => transferring dockerfile: 1.00kB                                           0.1s
=> [internal] load metadata for docker.io/library/python:3.10-slim              2.1s
=> [internal] load .dockerignore                                                  0.1s
=> => transferring context: 237B                                                 0.0s
=> [1/8] FROM docker.io/library/python:3.10-slim@sha256:e5300dc020a26a34a19337a57602955a2510e22abeb176edd6de6cd2 0.1s
=> => resolve docker.io/library/python:3.10-slim@sha256:e5300dc020a26a34a19337a57602955a2510e22abeb176edd6de6cd2 0.1s
=> [internal] load build context                                                0.0s
=> => transferring context: 624B                                                 0.0s
=> CACHED [2/8] WORKDIR /app                                                    0.0s
=> CACHED [3/8] RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt 0.0s
=> CACHED [4/8] COPY requirements.txt .                                          0.0s
=> CACHED [5/8] RUN pip install --no-cache-dir -r requirements.txt              0.0s
=> CACHED [6/8] COPY src/ ./src/                                                0.0s
=> CACHED [7/8] COPY models/ ./models/                                         0.0s
=> CACHED [8/8] RUN adduser --disabled-password --gecos '' appuser              0.0s
=> exporting to image                                                            0.6s
=> => exporting layers                                                            0.0s
=> => exporting manifest sha256:d7e7ea449ee98d0d4ecfd96af1bb294ed3564b23e0d2f32bc03785bb301eef00          0.1s
=> => exporting config sha256:7fed5c93afd38cdf3a7b7602722f1241685b350094ea035dafa17c032a378d3c          0.0s
=> => exporting attestation manifest sha256:c14445dc33fbaa59c386deaae6ee7745827d41646ea2d7db6b7423fd5ba934bf 0.1s
=> => exporting manifest list sha256:5be3a8ee2d9100d06120abba47cf987cb149b152a4f541d0ab1259725289d8a6      0.1s
=> => naming to docker.io/library/heart-disease-api:v1                          0.0s
=> => unpacking to docker.io/library/heart-disease-api:v1                       0.0s
(base) PS C:\gmr\projects\mtech_mlops_assignment_01_github\bits_sem3_mlops_001>
(base) PS C:\gmr\projects\mtech_mlops_assignment_01_github\bits_sem3_mlops_001> docker images heart-disease-api
IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
heart-disease-api:v1                5be3a8ee2d91                      2.54GB      752MB
(base) PS C:\gmr\projects\mtech_mlops_assignment_01_github\bits_sem3_mlops_001> |
```

Successful docker build of heart-disease-api:v1.

```
Windows PowerShell
(base) PS C:\gmr\projects\mtech_mlops_assignment_01_github\bits_sem3_mlops_001> $body = @{"age":67;sex:1;cp:4;trestbps:120;chol:229;fbs:0;restecg:2;thalach:129;exang:1;oldpeak:2.6;slope:2;ca:2;thal:7} | ConvertTo-Json
(base) PS C:\gmr\projects\mtech_mlops_assignment_01_github\bits_sem3_mlops_001> curl.exe http://localhost:8000/health
{"status":"healthy","model_loaded":true}
(base) PS C:\gmr\projects\mtech_mlops_assignment_01_github\bits_sem3_mlops_001> Invoke-RestMethod -Uri http://localhost:8000/predict -Method Post -Body $body -ContentType 'application/json' | ConvertTo-Json
{"prediction": 1,
 "probability": 0.9865,
 "risk_level": "High",
 "model_version": "1.0.0",
 "latency_ms": 47.74}
(base) PS C:\gmr\projects\mtech_mlops_assignment_01_github\bits_sem3_mlops_001> docker ps --filter "name=heart-disease-api"
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
9bb1ec773ac9   68a1af6f8214   "uvicorn src.api:app..." 9 hours ago    Up 9 hours    0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   heart-disease-api
(base) PS C:\gmr\projects\mtech_mlops_assignment_01_github\bits_sem3_mlops_001> |
```

Live /predict response from the running container.



Running docker containers

9. Kubernetes Deployment

The containerised API is deployed to a local Kubernetes cluster (Minikube) using two manifests under k8s/.

Deployment

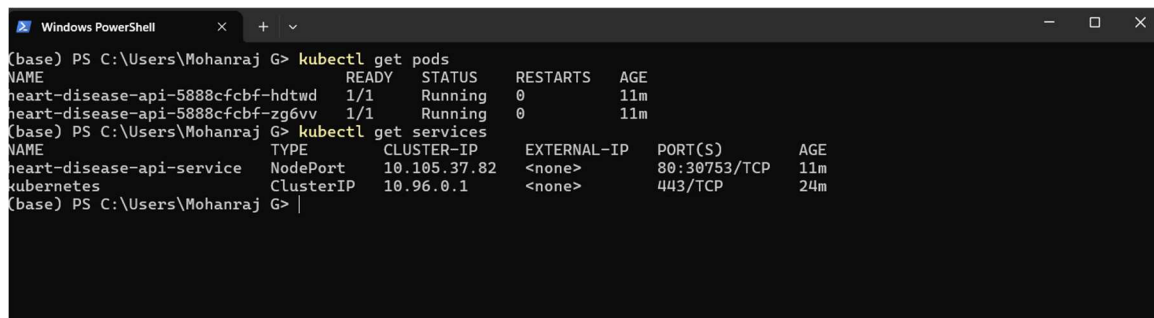
- **2 replicas** — for availability and basic load distribution.
- **Resource governance** — requests 256Mi/250m, limits 512Mi/500m per pod.
- **Liveness probe** — GET /health every 30s restarts a hung pod.
- **Readiness probe** — GET /health every 10s gates traffic until the model is loaded.

Service

A NodePort Service (heart-disease-api-service) maps cluster port 80 to the container's port 8000 and load-balances across the two pods. The

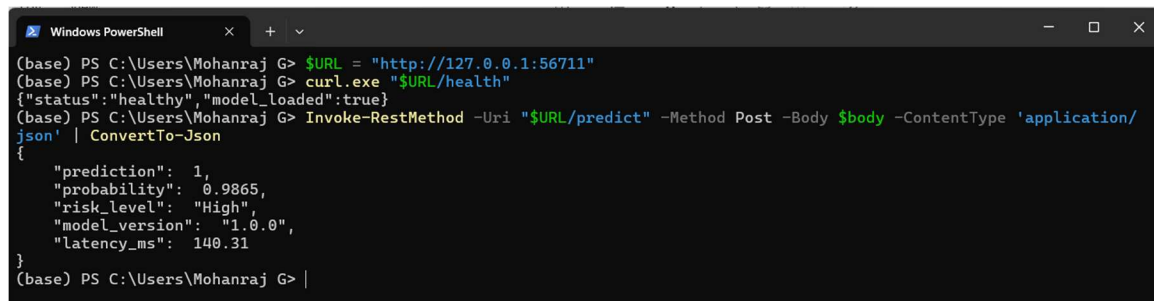
externally reachable URL is obtained with `minikube service heart-disease-api-service --url`.

```
minikube start --driver=docker --memory=4096 --cpus=2
minikube image load heart-disease-api:v1
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl get pods -w
```



```
(base) PS C:\Users\Mohanraj G> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
heart-disease-api-5888cfcbf-hdtwd   1/1     Running   0           11m
heart-disease-api-5888cfcbf-zg6vv   1/1     Running   0           11m
(base) PS C:\Users\Mohanraj G> kubectl get services
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
heart-disease-api-service  NodePort    10.105.37.82 <none>        80:30753/TCP     11m
kubernetes           ClusterIP   10.96.0.1    <none>        443/TCP          24m
(base) PS C:\Users\Mohanraj G> |
```

kubectl get pods — two replicas Running.



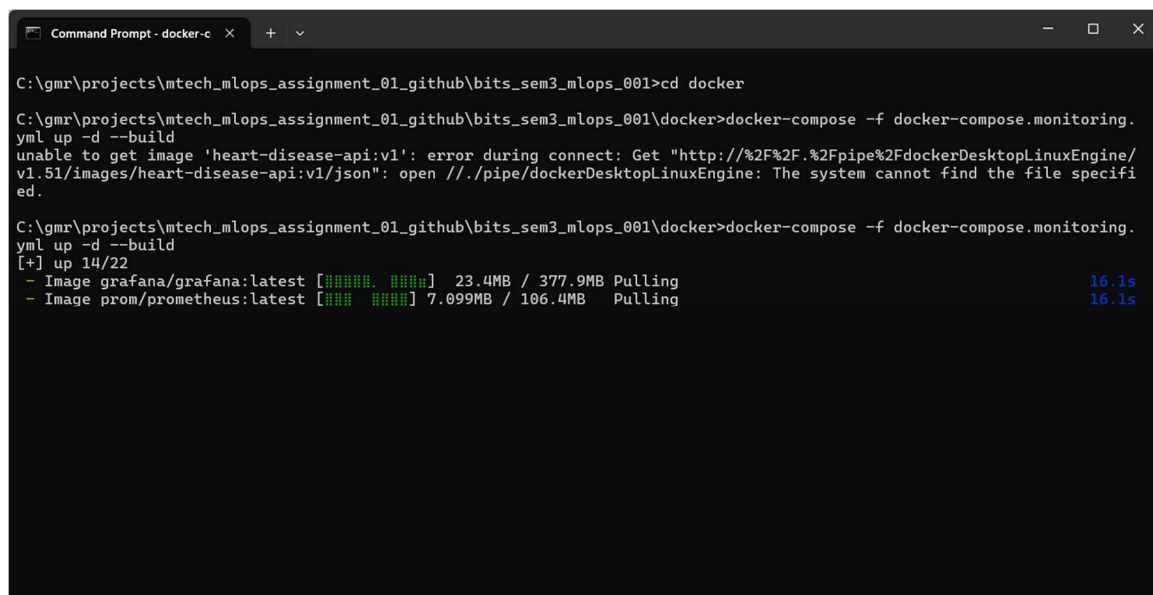
```
(base) PS C:\Users\Mohanraj G> $URL = "http://127.0.0.1:56711"
(base) PS C:\Users\Mohanraj G> curl.exe "$URL/health"
{"status":"healthy","model_loaded":true}
(base) PS C:\Users\Mohanraj G> Invoke-RestMethod -Uri "$URL/predict" -Method Post -Body $body -ContentType 'application/json' | ConvertTo-Json
{
  "prediction": 1,
  "probability": 0.9865,
  "risk_level": "High",
  "model_version": "1.0.0",
  "latency_ms": 140.31
}
(base) PS C:\Users\Mohanraj G> |
```

Figure: /predict served through the Minikube NodePort URL.

10. Monitoring & Conclusion

Observability Stack

The API auto-exposes Prometheus metrics at /metrics via prometheus-fastapi-instrumentator. The docker-compose.monitoring.yml stack brings up the API alongside Prometheus (which scrapes api:8000 every 15s) and Grafana. A Grafana dashboard visualises the golden signals: request rate, error rate, P95 latency, and in-progress requests. A load-test script (scripts/load_test.py) generates traffic so the dashboards populate.



```
Command Prompt - docker-c  X  +  v

C:\gmr\projects\mtech_mlops_assignment_01_github\bits_sem3_mlops_001>cd docker

C:\gmr\projects\mtech_mlops_assignment_01_github\bits_sem3_mlops_001\docker>docker-compose -f docker-compose.monitoring.
yaml up -d --build
unable to get image 'heart-disease-api:v1': error during connect: Get "http://%2F%2F.%2Fpipe%2FdockerDesktopLinuxEngine/
v1.51/images/heart-disease-api:v1/json": open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specifi
ed.

C:\gmr\projects\mtech_mlops_assignment_01_github\bits_sem3_mlops_001\docker>docker-compose -f docker-compose.monitoring.
yaml up -d --build
[+] up 14/22
- Image grafana/grafana:latest [#####] 23.4MB / 377.9MB Pulling 16.1s
- Image prom/prometheus:latest [####] 7.099MB / 106.4MB Pulling 16.1s
```

Command Prompt

#14 exporting attestation manifest sha256:bb65a0adf6da28a986f894e3774f8fda264085acb86c04987a2151d071dd703f
#14 exporting attestation manifest sha256:bb65a0adf6da28a986f894e3774f8fda264085acb86c04987a2151d071dd703f 0.1s done
#14 exporting manifest list sha256:68a1af6f82145c0f2bcfe6c740c1734fad29787b2ce7603883ac02de24d6ca67 0.1s done
#14 naming to docker.io/library/heart-disease-api:v1
#14 naming to docker.io/library/heart-disease-api:v1 done
#14 unpacking to docker.io/library/heart-disease-api:v1
#14 unpacking to docker.io/library/heart-disease-api:v1 77.5s done
#14 DONE 226.5s
[+] up 28/28
✔Image grafana/grafana:latest Pulled 217.9s
✔Image prom/prometheus:latest Pulled 102.3s
✔Image heart-disease-api:v1 Built 695.6s
✔Network docker_monitoring Created 0.2s
✔Volume docker_grafana-storage Created 0.8s
✔Container heart-disease-api Created 2.5s
✔Container prometheus Created 0.4s
✔Container grafana Created 6.3s

C:\gmr\projects\mtech_mlops_assignment_01_github\bits_sem3_mlops_001\docker>docker-compose -f docker-compose.monitoring.
yaml ps

NAME	PORTS	IMAGE	COMMAND	SERVICE	CREATED	STATUS
grafana		grafana/grafana:latest	"/run.sh"	grafana	50 minutes ago	Up 49 minutes
heart-disease-api	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	heart-disease-api:v1	"uvicorn src.api.app..."	api	50 minutes ago	Up 49 minutes (health)
prometheus	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp	prom/prometheus:latest	"/bin/prometheus --c..."	prometheus	50 minutes ago	Up 49 minutes

C:\gmr\projects\mtech_mlops_assignment_01_github\bits_sem3_mlops_001\docker>

localhost:8000/docs

Work

Heart Disease Risk Predictor API 1.0.0 OAS 3.1

/openapi.json

MLOps Assignment 01 — BITS Pilani AIMLCZG523

default

GET	/metrics	Metrics
GET	/	Root
GET	/health	Health Check
POST	/predict	Predict

Schemas

HTTPValidationError > Expand all object

PatientFeatures > Expand all object

```
localhost:8000/metrics
# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 2118.0
python_gc_objects_collected_total{generation="1"} 3622.0
python_gc_objects_collected_total{generation="2"} 1384.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 385.0
python_gc_collections_total{generation="1"} 34.0
python_gc_collections_total{generation="2"} 3.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="10",patchlevel="20",version="3.10.20"} 1.0
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 1.207278806e+09
# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 1.9176528e+08
# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1.78378482511e+09
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 8.129999999999999
# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds 17.0
# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1.048576e+06
# HELP http_requests_total Total number of requests by method, status and handler.
# TYPE http_requests_total counter
http_requests_total{method="GET",status="200"} 1.0
```

Lessons Learned

- **Package preprocessing with the model** — bundling the ColumnTransformer inside the Pipeline eliminated training/serving skew and made the API trivial.
- **Simplicity wins on small data** — a regularised linear model beat two ensembles on ROC-AUC and is cheaper to serve and easier to explain.
- **Reproducibility is a feature** — seeding everything and pinning dependencies meant the CI-trained model matched the local one exactly.
- **Gate early, gate loudly** — ordering CI as lint → test → build gives the fastest possible feedback on the cheapest failures.

Future Work

- Add data/model drift detection (e.g. Evidently) and automated retraining triggers.
- Introduce DVC for dataset and artifact versioning.
- Add a Helm chart and horizontal pod autoscaling for production-grade scaling.
- Push the image to a registry and deploy to a managed cluster (EKS/GKE/AKS).

Conclusion

This project delivers a complete, production-grade MLOps pipeline for heart-disease risk prediction: from a cleaned, well-understood dataset through a carefully-selected and tracked model, to a tested, containerised, orchestrated, and monitored service — all reproducible from a single command chain and continuously validated by CI/CD. The winning Logistic Regression model achieves a strong 0.9589 test ROC-AUC while remaining lightweight and interpretable, and the surrounding infrastructure demonstrates every stage of the modern ML lifecycle.

Repository: https://github.com/gmohanraj73/gmr_sem3_aimlczg523